



BACKEND



FRONTEND



API's

Table of Contents

<i>Introduction: [Project Name Here]</i>	3
Authors	3
Problem Statement	3
Solution	3
Implementation	3
<i>API Specification</i>	4
Authentication	5
Errors	6
Endpoints	7
Resource 1 (e.g. users).....	7

Introduction: Oblig 2 – REST APIs

Authors

- Adrian David Fonseca Malvik

Problem Statement

The Spanish Poker Dice Game from Oblig 1 is a simple 1v1 web game played in the browser, that runs on one device at the time. It does not allow for user registration, admins don't have a defined role with basic admin tools to manage the game, game stats are discarded after each round, no leaderboard, no ranking system, no tournament options, and no possibility for interaction between users.

Solution

Our suggested solution relies on the client-server architecture, and is to build a proper backend so it's possible for the frontend to build a full online experience and address the limitations of the current game. The server will be built using Node.js and Express.js. It will have a REST API and be connected to a database in MongoDB where all game data is stored.

Implementation

Backend follows MVCS architecture, separating concerns across routes, controllers, services and models. This keeps the code easier to maintain. Using Express.js for the server is time saving and code looks cleaner, not having to cram all the routes in a long if else conditional as you would if building the server in Node.js without Express.js. The REST API convention is the most popular way to write web APIs right now amongst developers, it is easy to read and is predictable. This combination of technologies supports handling CRUD operations.

API Specification

Authentication

All HTTP requests must include a "Role" header that says what type of user is making the request. The header accepts two values: "user" or "admin". If the header is missing or not recognized by the authentication middleware, then it is set to Role: "anonymous". Either way, the middleware stores the role in the request object, so all routes can tailor their respond to the correct user.

Errors

The following errors might be returned by the API.

HTTP status code	Error code	Description
401	UNKNOWN_USER	Login credentials don't match any existing user.
403	FORBIDDEN_ACCESS	Identified user is not of the required role to perform this action.
400	INVALID_REQUEST	Request has either missing required field(s), contains invalid field(s), and/or has incorrect format(s).
415	UNSUPPORTED_FILE_TYPE	Uploaded file type is not supported.
404	NOT_FOUND	Resource requested does not exist.
409	CONFLICT	Request conflicting with current state of data, such as duplicate entries or joining already full or started tournaments.
429	TOO_MANY_REQUESTS	Client sends too many requests within short amount of time.
500	INTERNAL_SERVER_ERROR	Unexpected error occurred on the server side.

Endpoints

Resource 1: users

URI (verb and route)	Description	Inputs	Outputs
POST /users	Create new user.	Headers: Content-Type: application/json. Body: username, email, password, dateOfBirth (all required).	201: { message: "User successfully registered" }
POST /users/login	Log in existing user.	Headers: Content-Type: application/json. Body: email, password (all required).	{ message: "Login successful", _id: ObjectId }
GET /users/:id	Fetch a user profile by id.	Route param: :id (required) – with the ObjectId of the user to fetch.	{ _id, username, email, dateOfBirth, elo, eloChangeThisWeek, role, trophies, recentGames: [last 10] }
PATCH /users/:id	Update a user profile by id.	Route param: :id (required) – with the ObjectId of the user to update. Headers: Content-Type: application/json. Body: email, password, dateOfBirth (all optional, only send what needs updating).	{ message: "User successfully updated" }
DELETE /users/:id	Deletes user by id. Requires admin role.	Route param: :id (required) – with ObjectId of the user to delete. Headers: Role: admin (required).	{ message: "User successfully deleted" }
GET /users	Fetch list of all users.	Query param: ?search=	Returns array

	Requires admin role. Possibility to filter by username.	(optional) – username. Headers: Role: admin.	with user objects.
GET /users/leaderboard	Fetch users with rankings by wins, win percentage or number of matches.	Query param: ?sort= (optional) – wins, winPercentage, totalMatches.	Returns sorted array with user objects. [{ _id, username, elo, wins, winPercentage, totalMatches }]
PATCH /users/:id/ban	Ban user by id. Requires admin role.	Route param: :id (required) – ObjectId of user to ban. Headers: Role: admin.	{ message: "User successfully banned" }
GET /users/activity	Fetch current platform activity.	None, visible to all roles.	{ activeGames, activeUsersThis Week, lastTenMatches: [{ _id, playerOne, playerTwo, winner, matchDate }] }

Resource 2: matches

URI (verb and route)	Description	Inputs	Outputs
POST /matches	Creates a new match.	Headers: Content-Type: application/json. Body: straightsAllowed, bestOf, timeControl (all required). tournamentId (optional) – include if match is part of tournament.	201: { message: "Match created successfully", _id: ObjectId }
GET /matches/:id	Fetches one match by id.	Route param: :id (required) . ObjectId of match to fetch.	{ _id, playerOne, playerTwo, straightsAllowed, bestOf, timeControl, winner, isDraw, tournament, matchDate }
GET /matches	Fetch list of all matches.	None, visible to all roles.	Array of match objects [{ _id, playerOne, playerTwo, winner, matchDate }]
PATCH /matches/:id	Update a match with results.	Route param: :id (required) – ObjectId of match to update. Headers: Content-Type: application/json. Body: winner, isDraw, matchDate (all optional, only send what needs updating).	{ message: "Match uupdated successfully" }
DELETE /matches/:id	Delete match by id. Requires admin role.	Route param: :id (required) – ObjectId of match to delete. Headers: Role: admin (required).	{ message: "Match deleted successfully" }

Resource 3: tournaments

URI (verb and route)	Description	Inputs	Outputs
POST /tournaments	Create a new tournament. Requires admin role.	Headers: Content-Type: application/json, Role: admin (all required). Body: title, description, straightsAllowed, bestOf, timeControl, startDate, endDate (all required). Trophy (optional).	{ message: "Tournament created successfully", _id: ObjectId }
GET /tournaments	Fetch list of all tournaments, both upcoming and past.	None, visible to all roles.	Returns array of tournament objects [{ _id, title, description, startDate, endDate, winner }]
GET /tournaments/:id	Fetch one tournament by id.	Route param: :id (required) – ObjectId of tournament to fetch.	{ _id, title, description, participants, matches, trophy, straightsAllowed, bestOf, timeControl, startDate, endDate, winner }
POST /tournaments/:id/join	Add user to a tournaments participant pool.	Route param: :id (required) – ObjectId of tournament to join. Headers: Role: user/admin (required, only one of the roles)	{ message: "Joined tournament successfully" }
DELETE /tournaments/:id	Delete tournament by id.	Route param: :id (required) – ObjectId of tournament to delete. Headers: Role: admin (required).	{ message: "Tournament deleted successfully" }

Resource 4: comments

URI (verb and route)	Description	Inputs	Outputs
POST /comments	Create new comment on match or tournament.	Headers: Content-Type: application/json, Role: user/admin (required). Body: content (required), matchId or tournamentId (one of them required)	{ message: "Comment created successfully", _id: ObjectId }
GET /comments	Fetch all comments. Can be filter by match or tournament.	Query param: ? matchId= or ? tournamentId= (optional).	Returns array of comment objects [{ _id, author, content, matchId, tournamentId, createdAt }]
DELETE /comments/:id	Delete a comment by id.	Route param: :id (required) – ObjectId of comment to delete. Headers: Role: admin (required)	{ message: "Comment deleted successfully" }

Resource 5: trophies

URI (verb and route)	Description	Inputs	Outputs
POST /trophies	Creates a new trophy.	Headers: Content-Type: application/json, Role: admin (all required). Body: title, image, tournament (all required).	{ message: "Trophy created successfully", _id: ObjectId }
GET /trophies/:id	Fetch one trophy by id.	Route param: :id (required) – ObjectId of trophy to fetch.	{ _id, title, image, tournament }